Modular Bus – Reference Architecture

Overview

This document defines a modular embedded platform centered around a generic CPU board and stackable daughter boards. The objective is to create a reusable hardware ecosystem optimized for **Rust + Embassy**, industrial communications, and IoT applications.

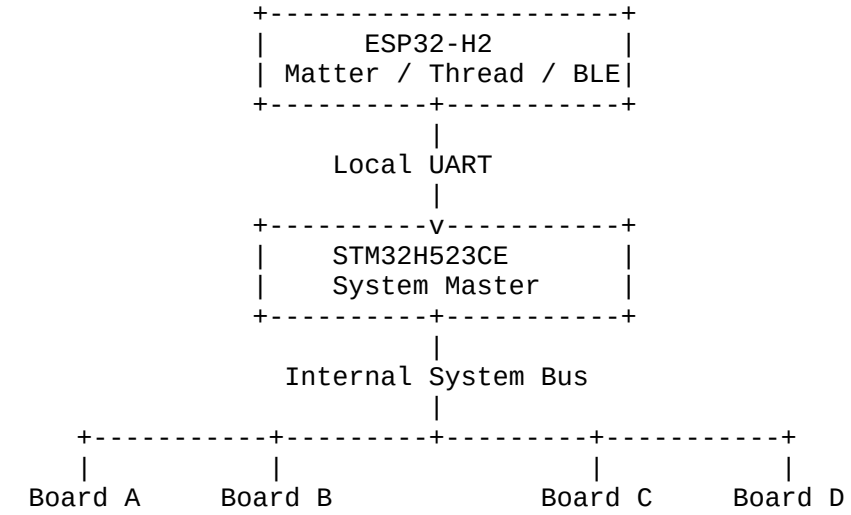
The architecture follows a simple principle:

- The **CPU board** provides computing resources.
- The **backplane** distributes only common infrastructure.
- Each **daughter board** implements its own peripherals locally.
- External field interfaces are implemented as dedicated interface boards.

Design Goals

- STM32H523CE as the main application processor
- ESP32-H2 as a dedicated Matter over Thread coprocessor
- Rust + Embassy software ecosystem
- Stackable daughter boards
- Generic CPU board
- Local high-speed peripherals
- External communication interfaces implemented as daughter boards
- Long-term hardware compatibility

System Architecture



The STM32H523CE is the system master.

The ESP32-H2 is a private wireless coprocessor and is **not visible** on the system bus.

Processor Responsibilities

STM32H523CE

- Main application
- Device manager
- Power management

- Backplane management
- CAN protocol stack
- Bootloader
- OTA updates
- Communication with ESP32-H2
- System supervision

ESP32-H2

- Matter
- Thread
- BLE commissioning
- Wireless networking

Communication with the STM32 is performed through a dedicated local UART.

Backplane Philosophy

The backplane distributes only shared infrastructure.
High-speed or board-specific interfaces remain local.

Distributed

- +5V
- +3.3V
- GND
- I²C
- Two UARTs
- CAN controller interface
- Shared interrupt
- Global reset
- Wake-up signal

Not Distributed

- SPI
- USB
- Ethernet
- SDIO
- Display interfaces
- RF interfaces
- CAN transceiver

J1 – System Bus

Pin	Signal	Description
1	GND	Ground
2	+5V	Main supply
3	+3V3	Logic supply
4	GND	Ground
5	I2C_SDA	Shared I²C bus

Pin	Signal	Description
6	I2C_SCL	Shared I²C bus
7	UART0_TX	Primary UART TX
8	UART0_RX	Primary UART RX
9	UART1_TX	Secondary UART TX
10	UART1_RX	Secondary UART RX
11	CAN_TX	FDCAN controller TX
12	CAN_RX	FDCAN controller RX
13	IRQ	Shared interrupt (open-drain recommended)
14	RESET	Global reset
15	WAKE	Wake-up signal
16	GND	Ground

STM32H523CE Mapping (J1)

Bus Signal	STM32H523CE Pin
I2C_SDA	PB9
I2C_SCL	PB8
UART0_TX	PA2 (USART2_TX)
UART0_RX	PA3 (USART2_RX)
UART1_TX	PA9 (USART1_TX)
UART1_RX	PA10 (USART1_RX)
CAN_TX	PA12 (FDCAN1_TX)
CAN_RX	PA11 (FDCAN1_RX)
IRQ	PC10
RESET	NRST
WAKE	PA0

Reserved MCU pins:

- PC13 → User LED
- PC14 / PC15 → Optional RTC crystal

J2 – General Purpose Expansion

Pin	Signal
1	GND
2	GPIO_A
3	GPIO_B
4	GPIO_C
5	GPIO_D
6	GPIO_E
7	GPIO_F
8	GPIO_G
9	GPIO_H
10	GPIO_I
11	GPIO_J
12	GPIO_K

Pin	Signal
13	ADC0
14	ADC1
15	PWM0
16	GND

STM32H523CE Mapping (J2)

Signal	STM32 Pin
GPIO_A	PB0
GPIO_B	PB1
GPIO_C	PB10
GPIO_D	PB11
GPIO_E	PB12
GPIO_F	PB13
GPIO_G	PB14
GPIO_H	PB15
GPIO_I	PC6
GPIO_J	PC7
GPIO_K	PC8
ADC0	PA4
ADC1	PA5
PWM0	PA8

Local STM32 ↔ ESP32-H2 Interface

The ESP32-H2 is connected only to the CPU board using a dedicated UART (preferably a UART instance **not exported** on the backplane, such as USART3 if available with a suitable pin assignment).

Typical connections:

STM32H523CE	ESP32-H2
Local UART_TX	RX
Local UART_RX	TX
GPIO	RESET
GPIO	HOST_WAKE

This interface is private and never appears on the backplane.

CAN Architecture

The backplane carries only the **CAN controller interface**.

It does **not** carry the differential CAN bus.

A dedicated daughter board converts the controller interface into a physical CAN network.

```
STM32H523CE
      |
 FDCAN Controller
      |
 CAN_TX / CAN_RX
      |
 Internal Backplane
```

```
      |
CAN Interface Board
      |
CAN-FD Transceiver
      |
CAN_H / CAN_L
      |
External Network
```

Different daughter boards may implement:

- Standard CAN
- CAN-FD
- Isolated CAN
- Automotive interfaces
- Industrial connectors

SPI Philosophy

SPI is always local.
Typical peripherals include:

- NOR Flash
- PSRAM
- TFT displays
- High-speed ADC
- DAC
- SD card
- Ethernet controller
- Specialized peripherals

SPI signals are never routed through the backplane.

Daughter Board Identification

Every intelligent daughter board should include a small I²C EEPROM for automatic discovery.
Suggested contents:

- Board type
- Hardware revision
- Manufacturer ID
- Serial number
- Unique identifier (EUI-48 or EUI-64)
- Configuration data

This allows the CPU board to enumerate installed modules automatically during boot.

Typical Daughter Boards

Examples include:

- Digital I/O
- Analog I/O
- Relay outputs
- Sensor interfaces
- Motor controllers

- CAN interface
- RS-485 interface
- Ethernet interface
- GNSS receiver
- LoRa radio
- Power supply modules

Each board implements only the peripherals it requires.

Software Architecture

STM32H523CE

- Embassy executor
- embassy-stm32
- Main application
- Device manager
- Bootloader
- CAN protocol stack

ESP32-H2

- Thread stack
- Matter stack
- BLE commissioning
- Local UART transport

Design Principles

- One CPU board controls the entire system.
- The backplane distributes only shared infrastructure.
- High-speed buses remain local to each board.
- External communication interfaces are implemented as dedicated daughter boards.
- The CPU board remains generic and reusable.
- The architecture is scalable and suitable for future expansion.

Advantages

- Clean and simple hardware architecture
- Minimal connector pin count
- Easy PCB routing
- Excellent scalability
- Native support for Rust + Embassy
- Generic and reusable CPU board
- Modular communication interfaces
- Easy manufacturing and maintenance
- Suitable for industrial control, IoT gateways, home automation, robotics, and embedded automation systems.